

assignment2_q3

May 8, 2026

1 Assignment 2 - RAG (and a bit of Reasoning)

Parts which require your interaction are marked with TODO:

```
[1]: # make sure we have the relevant dependencies
!pip3 install datasets sentence_transformers tqdm numpy
```

Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)

Requirement already satisfied: sentence_transformers in /usr/local/lib/python3.12/dist-packages (5.4.1)

Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (4.67.3)

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)

Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets) (3.29.0)

Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)

Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)

Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)

Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)

Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)

Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)

Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (2025.3.0)

Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (1.11.0)

Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets) (26.1)

Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets) (6.0.3)

Requirement already satisfied: transformers<6.0.0,>=4.41.0 in /usr/local/lib/python3.12/dist-packages (from sentence_transformers) (5.0.0)

Requirement already satisfied: torch>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from sentence_transformers) (2.10.0+cu128)

Requirement already satisfied: scikit-learn>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from sentence_transformers) (1.6.1)

Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from sentence_transformers) (1.16.3)

Requirement already satisfied: typing_extensions>=4.5.0 in /usr/local/lib/python3.12/dist-packages (from sentence_transformers) (4.15.0)

Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (3.13.5)

Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (1.4.3)

Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (0.28.1)

Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (0.24.2)

Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.4.7)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.13)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (2.5.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (2026.4.22)

Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.22.0->sentence_transformers) (1.5.3)

Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.22.0->sentence_transformers) (3.6.0)

Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence_transformers) (75.2.0)

Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence_transformers) (1.14.0)

Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence_transformers) (3.6.1)

Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence_transformers) (3.1.6)

Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from

```

torch>=1.11.0->sentence_transformers) (12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in
/usr/local/lib/python3.12/dist-packages (from
torch>=1.11.0->sentence_transformers) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from
torch>=1.11.0->sentence_transformers) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from
torch>=1.11.0->sentence_transformers) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in
/usr/local/lib/python3.12/dist-packages (from
torch>=1.11.0->sentence_transformers) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in
/usr/local/lib/python3.12/dist-packages (from
torch>=1.11.0->sentence_transformers) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in
/usr/local/lib/python3.12/dist-packages (from
torch>=1.11.0->sentence_transformers) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in
/usr/local/lib/python3.12/dist-packages (from
torch>=1.11.0->sentence_transformers) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in
/usr/local/lib/python3.12/dist-packages (from
torch>=1.11.0->sentence_transformers) (11.7.3.90)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in
/usr/local/lib/python3.12/dist-packages (from
torch>=1.11.0->sentence_transformers) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in
/usr/local/lib/python3.12/dist-packages (from
torch>=1.11.0->sentence_transformers) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in
/usr/local/lib/python3.12/dist-packages (from
torch>=1.11.0->sentence_transformers) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from
torch>=1.11.0->sentence_transformers) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in
/usr/local/lib/python3.12/dist-packages (from
torch>=1.11.0->sentence_transformers) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in
/usr/local/lib/python3.12/dist-packages (from
torch>=1.11.0->sentence_transformers) (1.13.1.3)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-
packages (from torch>=1.11.0->sentence_transformers) (3.6.0)

```

Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->torch>=1.11.0->sentence_transformers) (1.5.3)

Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41.0->sentence_transformers) (2025.11.3)

Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41.0->sentence_transformers) (0.22.2)

Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41.0->sentence_transformers) (0.24.0)

Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers<6.0.0,>=4.41.0->sentence_transformers) (0.7.0)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2026.1)

Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (2.6.1)

Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (1.4.0)

Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (26.1.0)

Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (1.8.0)

Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (6.7.1)

Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (0.4.1)

Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (1.23.0)

Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24.0->datasets) (4.13.0)

Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24.0->datasets) (1.0.9)

Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-

```

packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-
hub>=0.24.0->datasets) (0.16.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
packages (from python-dateutil>=2.8.2->pandas->datasets) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from
sympy>=1.13.3->torch>=1.11.0->sentence_transformers) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from
jinja2->torch>=1.11.0->sentence_transformers) (3.0.3)
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-
packages (from typer->huggingface-hub>=0.24.0->datasets) (8.3.3)
Requirement already satisfied: shellingham>=1.3.0 in
/usr/local/lib/python3.12/dist-packages (from typer->huggingface-
hub>=0.24.0->datasets) (1.5.4)
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-
packages (from typer->huggingface-hub>=0.24.0->datasets) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in
/usr/local/lib/python3.12/dist-packages (from typer->huggingface-
hub>=0.24.0->datasets) (0.0.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggingface-
hub>=0.24.0->datasets) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggingface-
hub>=0.24.0->datasets) (2.20.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-
packages (from markdown-it-py>=2.2.0->rich>=12.3.0->typer->huggingface-
hub>=0.24.0->datasets) (0.1.2)

```

Imagine your task is to build a question-answering (QA) system for a company. You are given a language model and have to create this product out of it. The requirements of the system need to adapt very quickly to the new data without training. For this, we will use Retrieval Augmented Generation (RAG). The company insists you use their in-house LM model trained on multiple tasks, a *flan-t5-small*. You can test its QA functionality by asking the question “*When ETH was founded?*”:

```

[2]: # There seems to be an issue with pipelines so we have this custom function.
from transformers import T5ForConditionalGeneration, T5Tokenizer

model = T5ForConditionalGeneration.from_pretrained("google/flan-t5-small")
tokenizer = T5Tokenizer.from_pretrained("google/flan-t5-small")
def vanilla_qa_pipe(text: str) -> str:
    inputs=tokenizer(text, return_tensors="pt")
    outputs = model.generate(**inputs, max_new_tokens=20)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

QUESTION = "When was ETH founded?"

```

```
vanilla_qa_pipe(f"QUESTION:{QUESTION} ANSWER:")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93:
```

```
UserWarning:
```

```
The secret `HF_TOKEN` does not exist in your Colab secrets.
```

```
To authenticate with the Hugging Face Hub, create a token in your settings tab  
(https://huggingface.co/settings/tokens), set it as secret in your Google Colab  
and restart your session.
```

```
You will be able to reuse this secret in all of your notebooks.
```

```
Please note that authentication is recommended but still optional to access  
public models or datasets.
```

```
warnings.warn(
```

```
config.json: 0.00B [00:00, ?B/s]
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a  
HF_TOKEN to enable higher rate limits and faster downloads.
```

```
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated  
requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and  
faster downloads.
```

```
model.safetensors: 0%| | 0.00/308M [00:00<?, ?B/s]
```

```
Loading weights: 0%| | 0/190 [00:00<?, ?it/s]
```

```
The tied weights mapping and config for this model specifies to tie  
shared.weight to lm_head.weight, but both are present in the checkpoints, so we  
will NOT tie them. You should update the config with `tie_word_embeddings=False`  
to silence this warning
```

```
generation_config.json: 0%| | 0.00/147 [00:00<?, ?B/s]
```

```
tokenizer_config.json: 0.00B [00:00, ?B/s]
```

```
spiece.model: 0%| | 0.00/792k [00:00<?, ?B/s]
```

```
tokenizer.json: 0.00B [00:00, ?B/s]
```

```
special_tokens_map.json: 0.00B [00:00, ?B/s]
```

```
[2]: '1897'
```

```
[3]: vanilla_qa_pipe(f"""  
    CONTEXT: ETH Zurich (German: Eidgenoessische Technische Hochschule_  
    ↪Zurich; English:  
    Federal Institute of Technology Zurich) is a public research university_  
    ↪in Zurich,  
    Switzerland. Founded in 1854 with the stated mission to educate_  
    ↪engineers and scientists,  
    the university focuses primarily on science, technology, engineering,_  
    ↪and mathematics. It
```

```

        consistently ranks among the top universities in the world and its 16
    ↪departments span a
        variety of disciplines and subjects.
    {QUESTION}
    ANSWER:",
    """,
)

```

[3]: '1854'

The first output is 1897, which is incorrect.

In the second example, the model was given context from which it could answer the question. So it gave the right answer. However, if you think of it, it was quite lucky that the context contained the answer it was looking for.

If you are not convinced, look at what happens when we change the question:

```

[4]: QUESTION = "When was EPFL founded?"
vanilla_qa_pipe(f"""
    CONTEXT: ETH Zurich (German: Eidgenoessische Technische Hochschule
    ↪Zurich; English:
        Federal Institute of Technology Zurich) is a public research university
    ↪in Zurich,
        Switzerland. Founded in 1854 with the stated mission to educate
    ↪engineers and scientists,
        the university focuses primarily on science, technology, engineering,
    ↪and mathematics. It
        consistently ranks among the top universities in the world and its 16
    ↪departments span a
        variety of disciplines and subjects.
    {QUESTION}
    ANSWER:",
    """,
)

```

[4]: '1854'

It would be nice if both federal institutes were founded in the same year, but they were not. EPFL was actually established in 1969. Now the obvious question would be, “Why not give the first paragraph of **EPFL’s** wikipedia page as context?

Well, yes, that would actually work. But for the company’s QA system, you cannot sit there looking at every query, searching for the relevant info and feed in.

You do know that customers will only ever ask about things the company works on, and all relevant info is in the company database.

Q1. So why don’t we just feed the model the entire database? (0.5 point)

A1. The model has a fixed context window (flan-t5-small accepts at most 512 input tokens), so

a real company knowledge base will not fit at all. Even on a large-context model, transformer self-attention is quadratic in the input length, so packing thousands of irrelevant documents into every prompt would make inference prohibitively slow and expensive. On top of that, irrelevant content actively *hurts* answer quality — the model gets distracted, attention mass is spread thinly, and useful evidence is drowned out. Restricting the context to the most relevant snippets is what makes RAG both feasible and accurate.

Okay, so we need something better. We earlier talked about how *you* cannot search for the context. But that does not mean *searching* was a bad idea. You could write code to automatically search the database and find the context. But what would you search for? Well, the only thing we have is the question, so might as well go with that.

```
[5]: # You only need to run this if you want to use nltk
import nltk
nltk.download('punkt_tab')
from nltk import word_tokenize
# You might need this
word_tokenize("Tokenize this. See what happens.")
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
```

```
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
```

```
[5]: ['Tokenize', 'this', '.', 'See', 'what', 'happens', '.']
```

```
[6]: class basic_search_engine:
    def __init__(self, dataset) -> None:
        self.dataset=list(dataset)
    def get_score(self, question_counts: dict, target: str) -> int:
        # count how many words are shared between the question and the target
        # if a word appears k times in the question and n times in the target
        # it should contribute k*n to the score.
        # Q2.1 TODO (1 points)
        target_tokens = word_tokenize(target.lower())
        target_counts: dict = {}
        for w in target_tokens:
            target_counts[w] = target_counts.get(w, 0) + 1
        score = 0
        for word, k in question_counts.items():
            if word in target_counts:
                score += k * target_counts[word]
        return score

    def search(self, question: str, count: int) -> list[str]:
        # return the top <count> contexts in terms of score
        # Q2.2 TODO (1 points)
        question_tokens = word_tokenize(question.lower())
        question_counts: dict = {}
        for w in question_tokens:
            question_counts[w] = question_counts.get(w, 0) + 1
```



```

        scores = [self.get_score(question_counts, t) for t in self.dataset]
        ranked = sorted(range(len(self.dataset)), key=lambda i: scores[i],
↪reverse=True)
        return [self.dataset[i] for i in ranked[:count]]

# Testing
data=["All cats can fly", "Some cats can fly planes", "Dogs fly higher than_
↪cats", "Dogs can not fly planes"]
question="Can dogs fly?"
search_engine=basic_search_engine(data)
search_engine.search(question, 2)

```

[6]: ['Dogs can not fly planes', 'All cats can fly']

If it worked with flying dogs, it should work with the real data. Test it out.

```

[7]: import tqdm
import numpy as np
from datasets import load_dataset
dataset = load_dataset("rajpurkar/squad")
context_set=set(dataset["validation"]["context"])
print("Total number of contexts is", len(context_set))
search_engine=basic_search_engine(context_set)

def metric_exact_match(ans_pred: str, ans_true: str) -> float:
    return (ans_pred == ans_true)*1.0

def metric_f1(ans_pred: str, ans_true: str) -> float:
    import collections
    ans_pred = ans_pred.lower().split()
    ans_true = ans_true.lower().split()
    common = collections.Counter(ans_pred) & collections.Counter(ans_true)
    num_same = sum(common.values())

    if num_same == 0:
        return num_same

    prec = num_same/len(ans_pred)
    recl = num_same/len(ans_true)
    return 2*prec*recl/(prec+recl)

exact_match_scores={}
f1_scores={}
exact_match_scores["search_1"]=[]
f1_scores["search_1"]=[]
for line in tqdm.tqdm(dataset["validation"].select(range(100))):
    question=line["question"]

```

```

contexts=search_engine.search(question, 1)
model_output=vanilla_qa_pipe(f"CONTEXT: {contexts[0]}\nQUESTION:
↪{question}\nANSWER:")
exact_match_scores["search_1"].append(max([metric_exact_match(model_output,
↪a) for a in line["answers"]["text"]]))
f1_scores["search_1"].append(max([metric_f1(model_output, a) for a in
↪line["answers"]["text"]]))
print()
print("Average Exact Match Score:", np.average(exact_match_scores["search_1"]))
print("Average F1 Score:", np.average(f1_scores["search_1"]))

```

README.md: 0.00B [00:00, ?B/s]

plain_text/train-00000-of-00001.parquet: 0%| | 0.00/14.5M [00:00<?, ?
↪B/s]

plain_text/validation-00000-of-00001.par(...): 0%| | 0.00/1.82M [00:00<?
↪, ?B/s]

Generating train split: 0%| | 0/87599 [00:00<?, ? examples/s]

Generating validation split: 0%| | 0/10570 [00:00<?, ? examples/s]

Total number of contexts is 2067

0%| | 0/100 [00:00<?, ?it/s]Token indices sequence length is longer
than the specified maximum sequence length for this model (681 > 512). Running
this sequence through the model will result in indexing errors
100%| | 100/100 [03:33<00:00, 2.13s/it]

Average Exact Match Score: 0.03

Average F1 Score: 0.07133333333333333

Well, Looks like that was neither fast, nor useful. The fact is, searching like this is not quite optimal.

Q3. So what are some issues with the search? Explain with insights from the ETH example (1 point)

A3.

- **It is purely lexical, not semantic.** The score only counts surface-form word overlap, so synonyms and paraphrases miss completely. For “When was EPFL founded?” any context that talks about something being *established*, *opened* or *created* will score zero on that informative word, while an unrelated paragraph that happens to share the verb “founded” (e.g., “Founded in 1854 ... ETH Zurich”) will rise to the top. That is exactly how the ETH paragraph ends up being retrieved as the “best” context for an EPFL question.
- **Common words dominate the score.** Stopwords like “when”, “was”, “the”, “is” appear in almost every context and contribute heavily to the score even though they carry no information about the question. There is no down-weighting of frequent, uninformative tokens

(which is precisely the gap that TF-IDF fills).

- **Length bias.** A long Wikipedia paragraph naturally contains more tokens, so its expected overlap with any question is higher. Useful short contexts are systematically penalised.
- **No notion of named entities or topicality.** The word “ETH” or “EPFL” in the query should match the *topic* of the context, but it is treated identically to a generic verb like “founded”. A context about EPFL might score lower than a context about ETH that happens to repeat several stopwords from the question.
- **Inefficient.** The score is recomputed against every document with a Python loop, which scales as $O(|DB| \times |\text{context}|)$ per query and is a non-starter on a real company database.

Searching a database like this is a non-trivial task and a whole research field of Information Retrieval is devoted to it.

For now, we will pick two off-the-shelf methods, TF-IDF Vectorization and Embedding with Bert.

You might want to look up what these do. Then answer all the following in 1-3 sentences (0.5 point each)

Q4.1. What is TF-IDF?

A4.1. TF-IDF (Term Frequency – Inverse Document Frequency) is a classic word-weighting scheme that scores each (word, document) pair as $\text{tf}(\mathbf{w}, \mathbf{d}) * \text{idf}(\mathbf{w})$, where tf measures how often the word appears in the document and $\text{idf} = \log(N / \text{df}(\mathbf{w}))$ down-weights words that appear in many documents. The result is that words which are frequent in *this* document but rare across the *whole* corpus get the highest weight, so a TF-IDF vector emphasises a document’s distinctive terms while suppressing stopwords.

Q4.2. What is the benefit of vectorisation?

A4.2. Once every document and every query is represented as a fixed-size numeric vector, similarity becomes a single matrix–vector dot product, which is orders of magnitude faster than comparing strings token by token and is trivially batchable on a GPU. It also lets us reuse the whole geometric / linear-algebra toolbox – cosine similarity, nearest-neighbour indexes (FAISS, HNSW), clustering, dimensionality reduction – none of which are available on raw text.

Q4.3. What is a sentence embedding?

A4.3. A sentence embedding is a single dense vector that summarises the *meaning* of a whole sentence, typically produced by a transformer (e.g. SBERT) and trained so that semantically similar sentences land close together in the embedding space, regardless of whether they share any surface words. This lets a search engine retrieve “EPFL was established in 1969” for the query “When was EPFL founded?” even though the two sentences have almost no token overlap.

Now complete the code below. We have already filled in the initialization

```
[8]: from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer(max_features=768, norm=None)

class tfidf_search_engine:
    def __init__(self, dataset) -> None:
        self.vectorizer = TfidfVectorizer(max_features=768, norm=None)
        self.dataset=list(dataset)
```

```

        self.kb_vectorized = self.vectorizer.fit_transform([x for x in self.
↪dataset])

    def search(self, question: str) -> list[str]:
        # return the top contexts in terms of tf-idf score
        # we use cosine similarity, so score(a,b)=vector(a).vector(b)
        # you can use the "transform" function of self.vectorizer
        # q5. TODO (1 point)
        q_vec = self.vectorizer.transform([question])           # sparse: [1,
↪vocab]

        scores = (self.kb_vectorized @ q_vec.T).toarray().ravel() # [n_dataset]
        best_idx = int(np.argmax(scores))
        return self.dataset[best_idx]

# Testing
data=["All cats can fly", "Some cats can fly planes", "Dogs fly higher than
↪cats", "Dogs can not fly planes"]
question="Can dogs fly?"
search_engine=tfidf_search_engine(data)
search_engine.search(question)

```

[8]: 'Dogs can not fly planes'

```

[9]: exact_match_scores["search_tfidf"]=[]
f1_scores["search_tfidf"]=[]
tfidf_engine=tfidf_search_engine(context_set)
for line in tqdm.tqdm(dataset["validation"].select(range(100))):
    question=line["question"]
    contexts=tfidf_engine.search(question)
    model_output=vanilla_qa_pipe(f"CONTEXT: {contexts}\nQUESTION:
↪{question}\nANSWER:")
    exact_match_scores["search_tfidf"].
↪append(max([metric_exact_match(model_output, a) for a in
↪line["answers"]["text"]]))
    f1_scores["search_tfidf"].append(max([metric_f1(model_output, a) for a in
↪line["answers"]["text"]]))
print()
print("Average Exact Match Score:", np.
↪average(exact_match_scores["search_tfidf"]))
print("Average F1 Score:", np.average(f1_scores["search_tfidf"]))

```

100%| | 100/100 [00:35<00:00, 2.80it/s]

Average Exact Match Score: 0.05

Average F1 Score: 0.12533333333333332

```
[10]: from sentence_transformers import SentenceTransformer

class sbert_search_engine:
    def __init__(self, dataset) -> None:
        self.vectorizer = SentenceTransformer("bert-base-nli-mean-tokens").cuda()
        self.dataset=list(dataset)
        self.kb_vectorized = self.vectorizer.encode(self.dataset)
    def search(self, question: str) -> list[str]:
        # return the top contexts in terms of bert-similarity
        # we use cosine similarity, so score(a,b)=vector(a).vector(b)
        # you can use the "encode" function of self.vectorizer
        # q6. TODO (1 point)
        q_vec = self.vectorizer.encode(question)           # [dim]
        scores = self.kb_vectorized @ q_vec                # [n_dataset]
        best_idx = int(np.argmax(scores))
        return self.dataset[best_idx]

# Testing
data=["All cats can fly", "Some cats can fly planes", "Dogs fly higher than_
↪cats", "Dogs can not fly planes"]
question="Can dogs fly?"
search_engine=sbert_search_engine(data)
search_engine.search(question)
```

```
modules.json: 0%|          | 0.00/229 [00:00<?, ?B/s]
config_sentence_transformers.json: 0%|          | 0.00/122 [00:00<?, ?B/s]
README.md: 0.00B [00:00, ?B/s]
sentence_bert_config.json: 0%|          | 0.00/53.0 [00:00<?, ?B/s]
config.json: 0%|          | 0.00/625 [00:00<?, ?B/s]
model.safetensors: 0%|          | 0.00/438M [00:00<?, ?B/s]
Loading weights: 0%|          | 0/199 [00:00<?, ?it/s]

BertModel LOAD REPORT from: sentence-transformers/bert-base-nli-mean-tokens
Key          | Status      | |
-----+-----+---+
embeddings.position_ids | UNEXPECTED | |

Notes:
- UNEXPECTED      :can be ignored when loading from different task/architecture;
not ok if you expect identical arch.

tokenizer_config.json: 0%|          | 0.00/399 [00:00<?, ?B/s]
vocab.txt: 0.00B [00:00, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
```

```
added_tokens.json: 0%|          | 0.00/2.00 [00:00<?, ?B/s]
special_tokens_map.json: 0%|          | 0.00/112 [00:00<?, ?B/s]
config.json: 0%|          | 0.00/190 [00:00<?, ?B/s]
```

```
[10]: 'Dogs fly higher than cats'
```

```
[11]: exact_match_scores["search_sbert"]=[]
sbert_engine=sbert_search_engine(context_set)
f1_scores["search_sbert"]=[]
for line in tqdm.tqdm(dataset["validation"].select(range(100))):
    question=line["question"]
    contexts=sbert_engine.search(question)
    model_output=vanilla_qa_pipe(f"CONTEXT: {contexts}\nQUESTION:
    ↳{question}\nANSWER:")
    exact_match_scores["search_sbert"].
    ↳append(max([metric_exact_match(model_output, a) for a in
    ↳line["answers"]["text"]]))
    f1_scores["search_sbert"].append(max([metric_f1(model_output, a) for a in
    ↳line["answers"]["text"]]))
print()
print("Average Exact Match Score:", np.
    ↳average(exact_match_scores["search_sbert"]))
print("Average F1 Score:", np.average(f1_scores["search_sbert"]))
```

```
Loading weights: 0%|          | 0/199 [00:00<?, ?it/s]
```

```
BertModel LOAD REPORT from: sentence-transformers/bert-base-nli-mean-tokens
```

```
Key | Status | |
-----+-----+---
embeddings.position_ids | UNEXPECTED | |
```

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture;
not ok if you expect identical arch.

```
100%|          | 100/100 [00:24<00:00, 4.04it/s]
```

```
Average Exact Match Score: 0.17
```

```
Average F1 Score: 0.18266666666666664
```

You would note that the scores are still not that high. Perhaps because the similarity metrics are not perfect.

Q7. Calculate how often the top scoring context is in fact the correct context for a) TF-IDF and b) sBert Score (you would have to write some code for this. Only work with the first 100 validation questions) (1.5 point)

A7. Running the cell below on the first 100 SQuAD validation questions gives:

- **TF-IDF top-1 accuracy: 0.00** (0 / 100)
- **SBERT top-1 accuracy: 0.12** (12 / 100)

Both numbers are surprisingly low, but they are consistent with what we saw in the QA experiments (cell 17 EM = 0.05 with TF-IDF vs. cell 19 EM = 0.17 with SBERT) and they explain *why* the QA scores are so weak: the retriever almost never lands on the gold context.

Two effects combine to produce these numbers:

1. **The vectorizer is configured with norm=None.** Without L2-normalization the score is just a raw dot product, which is dominated by document length and by the absolute frequency of the matched terms. As a result the TF-IDF retriever returns essentially the same handful of long contexts for almost every question, and on a 2067-context knowledge base that handful never coincides with the per-question gold context — hence the 0/100. Switching to `norm="l2"` or applying cosine normalization manually would already lift this score substantially.
2. **The SBERT checkpoint is bert-base-nli-mean-tokens**, an older NLI-trained encoder that was not specifically optimized for asymmetric question→passage retrieval. It still beats TF-IDF on this task — confirming the qualitative point that dense semantic embeddings capture paraphrases TF-IDF cannot — but a modern retrieval-tuned model such as `all-MiniLM-L6-v2` or `multi-qa-MiniLM-L6-cos-v1` would push this number much higher.

The qualitative conclusion stands: SBERT > TF-IDF (0.12 > 0.00), and the gap is the headroom that downstream QA accuracy is missing.

```
[12]: # write your code for Q7 here
# The 'correct' context for the question dataset["validation"]["question"][i] is
# dataset["validation"]["context"][i]
matches_tf_idf=0
matches_sbert=0
for i in tqdm.tqdm(range(100)):
    if dataset["validation"]["context"][i]==tfidf_engine.
↪search(dataset["validation"]["question"][i]):
        matches_tf_idf+=1
    if dataset["validation"]["context"][i]==sbert_engine.
↪search(dataset["validation"]["question"][i]):
        matches_sbert+=1
print("TF-IDF:", matches_tf_idf/100)
print("SBERT:", matches_sbert/100)
```

```
100%|          | 100/100 [00:02<00:00, 49.54it/s]
```

```
TF-IDF: 0.0
```

```
SBERT: 0.12
```

Q8. What can we do to account for imperfect similarity metrics? (Hint: if you have been following the code, there was already a hint somewhere) (0.5 point)

A8. The hint is the `count` argument of `basic_search_engine.search`: instead of returning only the single highest-scoring context, retrieve the **top-k** candidates and feed all of them to the QA

model in one prompt. As long as the correct context is anywhere in the top-k (which, per Q7, happens far more often than it is exactly rank-1), the model can attend to it and produce the right answer. This is the standard “retrieve-then-read” recipe used by real RAG systems, and it can be combined with re-ranking (run an expensive cross-encoder on the k candidates) to push the right context to the very top.

You may have noticed that sometimes model can give the right answer despite having the wrong context. But the reverse can also be true.

Q9. For each of the following examples, explain why the model is getting the answer wrong (0.5 point each)

```
[13]: context="The Berlin Wall (German: Berliner Mauer) was a guarded concrete
↳ barrier that encircled West Berlin from 1961 to 1989, separating it from
↳ East Berlin and the German Democratic Republic (GDR; East Germany).
↳ Construction of the Berlin Wall was commenced by the government of the GDR
↳ on 13 August 1961. It included guard towers placed along large concrete
↳ walls,[4] accompanied by a wide area (later known as the \"death strip\")
↳ that contained anti-vehicle trenches, beds of nails and other defenses. The
↳ primary intention for the Wall's construction was to prevent East German
↳ citizens from fleeing to the West"
question="For many years did the Berlin wall stand?"
vanilla_qa_pipe(f"CONTEXT: {context}\nQUESTION:{question}\nANSWER:")
```

[13]: 'a guarded concrete barrier'

A9.1: The model returned “**a guarded concrete barrier**”, which is the very first noun phrase the context uses to predicate something about the Berlin Wall (“*The Berlin Wall ... **was a guarded concrete barrier** that encircled West Berlin from 1961 to 1989*”). The reason this happens is that the question is grammatically malformed — it should read “*For **how** many years did the Berlin Wall stand?*”. Without the wh-word “how” the prompt is no longer recognisable as a duration question; flan-t5-small parses “*... did the Berlin Wall stand?*” as a generic “what was/did the Berlin Wall ...?” query and falls back to extracting the first salient predicate from the context. Even if the question were well-formed, the correct answer (28) is not present in the context as a span — it has to be derived as 1989 − 1961, and a 77M-parameter encoder–decoder like flan-t5-small is poor at multi-step arithmetic on dates. So this is a combined failure: (i) a defective question that misleads the parser into treating it as a definitional query, and (ii) a small model that cannot perform the date subtraction it would need even if the parse were right.

```
[14]: context="Joseph Robinette Biden Jr.(born November 20, 1942) is an American
↳ politician who was the 46th president of the United States from 2021 to 2025.
↳ A member of the Democratic Party, he represented Delaware in the United
↳ States Senate from 1973 to 2009 and also served as the 47th vice president
↳ under President Barack Obama from 2009 to 2017."
question="Who was the president of United States during 2021-2025?"
vanilla_qa_pipe(f"CONTEXT: {context}\nQUESTION:{question}\nANSWER:")
```

[14]: 'President Barack Obama'

A9.2 The correct answer (“Joseph Robinette Biden Jr.” or simply “Biden”) *is* present in the context,

but it appears at the very start, syntactically detached from the phrase “*from 2021 to 2025*”; the model has to bind the subject of the first sentence (“Joseph Robinette Biden Jr.”) to the role description in the relative clause (“who was the 46th president of the United States from 2021 to 2025”). The context then immediately introduces a second president by name – “**President** Barack Obama” – explicitly preceded by the title “President”, which is a much stronger surface cue for an extractive QA model than the more distant relative-clause construction that mentions Biden. `flan-t5-small` therefore latches onto the closest “President X” pattern and answers “Barack Obama”. This is a classic distractor-induced failure: even with the correct context, a small model picks the answer with the strongest local lexical signal rather than the one supported by the semantics.

#END OF ASSIGNMENT